

Universitatea de Vest din Timișoara

Facultatea de Matematică și Informatică

Analiza comparativă a algoritmilor de sortare

Lucrare de laborator / proiect la Scriere Academica

Student: Minciuna David-Florian

Grupă: 4

Coordonator: Adrian Craciun

An universitar: 2025–2026

Timișoara

Rezumat

This paper presents a comparative analysis of classical sorting algorithms: Bubble Sort, Selection Sort, Insertion Sort, and Quick Sort. The performance is evaluated experimentally on different types of input data.

Keywords: sorting, quicksort, complexity, algorithms

1 Introducere

Sortarea datelor este o operație fundamentală în informatică, utilizată în numeroase aplicații. Alegerea algoritmului optim depinde de natura datelor și dimensiunea acestora.

2 Problema și soluția

Problema constă în ordonarea unei liste de numere întregi.

Algoritmii analizați:

- Bubble Sort
- Selection Sort
- Insertion Sort
- Quick Sort

Complexități:

- $O(n^2)$: Bubble, Selection, Insertion
- $O(n \log n)$: QuickSort (medie)

3 Modelare și implementare

Testele au fost realizate pe:

- liste aleatoare
- liste sortate
- liste invers sortate
- liste aproape sortate

3.1 Mediul experimental

Experimentele au fost realizate pe următorul sistem:

- Placă video: NVIDIA RTX 2060
- Procesor: Intel Core i7-9700F
- Memorie RAM: 16 GB
- Stocare: SSD 1TB
- Sistem de operare: Windows 11

3.2 Implementare exemplu

```
def quickSort(l):  
    if len(l)<=1:  
        return l  
    pivot=l[0]  
    st=[]  
    dr=[]  
    for i in l[1:]:  
        if i <= pivot:  
            st.append(i)  
        else:  
            dr.append(i)  
    return quickSort(st)+[pivot]+quickSort(dr)
```

4 Studii de caz

Rezultatele experimentale (exemplu):

Tip lista	Bubble	Selection	Insertion	Quick
Aleatoare	0.45	0.40	0.20	0.01
Sortata	0.01	0.40	0.01	0.02
Invers	0.50	0.42	0.35	0.03
Aproape	0.10	0.38	0.02	0.01

Observații:

- QuickSort este cel mai eficient în majoritatea cazurilor
- Insertion Sort este optim pentru liste aproape sortate
- Bubble Sort este ineficient

5 Compararea cu literatura

Algoritmul QuickSort, propus de C.A.R. Hoare în 1960, este cunoscut pentru eficiența sa în practică. Literatura confirmă superioritatea algoritmilor divide-et-impera față de cei de complexitate quadratică.

Rezultatele obținute sunt în concordanță cu teoria.

6 Concluzii

Lucrarea evidențiază diferențele majore între algoritmii analizați.

Dirrecții viitoare:

- implementarea MergeSort și HeapSort
- analiza memoriei

7 Bibliografie

1. C. A. R. Hoare, *Quicksort*, Communications of the ACM, 1961. (autorul algoritmului QuickSort)
2. Donald E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, Addison-Wesley, 1998. (analiza și formalizarea algoritmilor Bubble Sort, Selection Sort și Insertion Sort)
3. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, *Introduction to Algorithms*, MIT Press, 2009. (prezentare și analiză modernă a algoritmilor de sortare utilizați)

8 Anexe

8.1 Cod sursă complet

```
import time
import random
import sys
sys.setrecursionlimit(10**6)

def bubbleSort(l):
    l = l.copy()
    n=len(l)
```

```

    for i in range(n):
        swapped=False
        for j in range(0,n-i-1):
            if l[j]>l[j+1]:
                l[j],l[j+1]=l[j+1],l[j]
                swapped=True
        if not swapped:
            break
    return l

def selectionSort(l):
    l = l.copy()
    n=len(l)
    for i in range(n-1):
        minim=i
        for j in range(i+1,n):
            if l[j]<l[minim]:
                minim=j
        l[i],l[minim]=l[minim],l[i]
    return l

def insertionSort(l):
    l = l.copy()
    n=len(l)
    for i in range(1,n):
        key=l[i]
        j=i-1
        while j>=0 and key<l[j]:
            l[j+1]=l[j]
            j-=1
        l[j+1]=key
    return l

def quickSort(l):
    l = l.copy()
    if len(l)<=1:
        return l
    pivot=l[len(l)//2]
    st=[]
    dr=[]
    egal=[]
    for i in l:
        if i<pivot:

```

```
        st.append(i)
    elif i>pivot:
        dr.append(i)
    else:
        egal.append(i)
    return quickSort(st)+egal+quickSort(dr)
```

8.2 Observații tehnice

Fiecare algoritm operează pe o copie a listei pentru a evita modificarea datelor inițiale. Acest lucru asigură corectitudinea măsurătorilor și independența testelor.